

Senior DevOps / SRE

Interview Mastery Guide

20 Toughest Questions — Deep Answers + Tactics

Kubernetes • CI/CD • Cloud • Security • Observability • Incident Handling

Prepared for: Sanket | April 2026

SECTION 1 — Kubernetes & Containers

Q1

Pod is Running but returning 503 — how do you debug at network, service, and ingress level?

Step-by-step debug ladder:

1. **Pod level:** `kubectl describe pod` — check `readinessProbe` status. 503 most often means pod is running but NOT ready.
2. **Service selector:** `kubectl get endpoints` — if empty, the label selector doesn't match pod labels. Fix: `kubectl get pod --show-labels`
3. **Service port mapping:** Verify `targetPort` in Service matches `containerPort` in Pod spec. A mismatch causes silent drops.
4. **DNS resolution:** `kubectl exec -- nslookup .svc.cluster.local` — rules out CoreDNS issues.
5. **Direct curl to pod IP:** `kubectl get pod -o wide`, then `curl :/health` — bypasses Service entirely.
6. **Ingress rules:** `kubectl describe ingress` — check path, host, and backend service name + port.
7. **Ingress controller logs:** `kubectl logs -n ingress-nginx deploy/ingress-nginx-controller` — reveals upstream connect failures.
8. **NetworkPolicy:** `kubectl get networkpolicy -A` — a deny-all policy blocking ingress controller -> pod traffic is a common hidden culprit.

Key commands:

```
kubectl get endpoints -n
kubectl describe ingress -n
kubectl exec -it -- curl -v http://:/
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller --tail=100
```

Interviewer trap: They expect you to go beyond 'kubectl describe pod'. Mention endpoints, NetworkPolicy, and ingress controller logs — that's the senior signal.

Q2

How does Kubernetes scheduling work internally, and what are common causes of scheduling failures?

Scheduler pipeline (3 phases):

- Filter (Predicates):** Eliminates nodes that can't run the pod: insufficient CPU/memory, taint not tolerated, node affinity not met, volume zone mismatch, `PodFitsHostPorts`.
- Score (Priorities):** Ranks remaining nodes: `LeastRequestedPriority` (spread load), `NodeAffinityPriority`, `InterPodAffinityPriority`, `ImageLocalityPriority`.
- Bind:** Selected node is written to the pod's `spec.nodeName` via the API server. Kubelet on that node picks it up and starts the pod.

Common scheduling failures:

- 0/3 nodes are available: Insufficient CPU/memory — requests too high vs node capacity
- 0/3 nodes available: node(s) had taint {key:value}, pod didn't tolerate — add toleration or taint key fix
- `PodAffinity` / `PodAntiAffinity` deadlock — services requiring co-location but conflicting rules
- PersistentVolume zone mismatch — EBS volume in us-east-1a, node in us-east-1b
- Resource quota exceeded in namespace — `kubectl describe resourcequota -n`
- Pending forever + `ImagePullBackOff` — image missing + scheduling succeeded but container fails

Pro tip: 'kubectl describe pod ' always shows 'Events' section with exact scheduler rejection reason. Always start there.

Q3

How do you design zero-downtime deployments for stateful applications in Kubernetes?

Stateful vs Stateless — key difference:

Stateless apps: rolling update is safe. Stateful apps (databases, Kafka, Zookeeper) have ordered startup/shutdown requirements, shared storage, and quorum concerns.

Design decisions:

StatefulSet with orderedReady: Default podManagementPolicy. Pod-0 must be Running+Ready before Pod-1 starts. Safe for leader-follower DBs.

PodDisruptionBudget (PDB): spec.minAvailable: 2 ensures at least 2 replicas stay up during voluntary disruption (node drain, rolling update).

preStop lifecycle hook: Add preStop: exec: command: ['sleep','15'] to allow in-flight requests and connection drain before SIGTERM.

ReadinessGate / custom probes: Only mark pod ready AFTER replication lag < threshold (for MySQL replicas etc.).

Init containers for migration: Run DB schema migration (Flyway/Liquibase) in init container — ensures migration runs before app starts.

Persistent Volume claim retention: Set persistentVolumeClaimRetentionPolicy: whenDeleted: Retain to avoid accidental data loss on pod delete.

ApnaKart connection: Your ApnaKart project uses StatefulSets for RDS + Flyway in init containers. Mention that directly — it's real hands-on evidence.

Q4

Explain how CNI plugins work and how cross-node pod communication happens.

What CNI does:

Container Network Interface is a spec. When kubelet creates a pod, it calls the CNI plugin binary to: (1) create a virtual ethernet pair (veth), (2) place one end in the pod netns, (3) assign an IP from the pod CIDR, (4) set up routing rules on the node.

Cross-node pod traffic flow (example: Calico/VPC CNI):

- Pod A (node1, 10.0.1.5) sends packet to Pod B (node2, 10.0.2.7)
- Kernel routing on node1: 10.0.2.0/24 via node2's IP (learned via BGP in Calico or VPC route table in AWS VPC CNI)
- Packet leaves node1 NIC → travels to node2 via underlying network
- node2 kernel receives, CNI routes to veth pair of Pod B
- AWS VPC CNI: pods get real VPC IPs, no overlay needed — direct routing. Most efficient.
- Calico: BGP between nodes. Each node advertises its pod CIDR. No encapsulation overhead in native mode.
- Flannel: VXLAN overlay — encapsulates pod packets in UDP. Simpler but adds overhead.

Kubernetes network model rules (must know):

- Every pod gets a unique IP (no NAT between pods on same cluster)
- Pods can communicate with all other pods without NAT
- Nodes can communicate with all pods without NAT
- The IP a pod sees as its own is the same IP others use to reach it

AWS EKS uses VPC CNI by default — each pod gets an ENI secondary IP. This is what your ApnaKart cluster uses. Mention it!

Q5

How do you debug intermittent pod restarts with no clear logs?

Systematic investigation:

OOMKilled check: `kubectl describe pod` — look for 'OOMKilled' in Last State. `kubectl top pod` — check memory usage vs limits.

Previous container logs: `kubectl logs --previous` — logs from the crashed container, not current one. Critical step most forget.

Exit code analysis: Exit 137 = OOMKill. Exit 1 = app crash. Exit 143 = SIGTERM (graceful stop). Exit 139 = segfault.

Liveness probe tuning: Misconfigured liveness probe (too aggressive initialDelaySeconds) kills healthy pods. Check probe thresholds.

Node pressure events: `kubectl describe node` — look for MemoryPressure, DiskPressure eviction events.

Metrics timeline: Use Prometheus/Grafana: graph `container_memory_usage_bytes` and `container_oom_events_total` over time.

Datadog APM traces: Correlate restart timestamps with APM traces to identify if a specific endpoint triggers the crash.

```
kubectl logs --previous
```

```
kubectl describe pod | grep -A5 'Last State'
```

```
kubectl get events --sort-by='.lastTimestamp' -n
```

Intermittent restarts with no logs = strong OOMKill signal. Your Datadog APM in ApnaKart can correlate restart times with memory metrics — mention this.

SECTION 2 — CI/CD & Performance

Q6

CI pipeline builds 50 images and takes 20 minutes — how do you reduce it to <5 minutes?

Optimization strategy (layered approach):

- 1. Docker layer caching:** Use `--cache-from` to pull previous image before build. Most impactful single change. Works with BuildKit.
- 2. Parallel matrix builds:** GitHub Actions matrix strategy / Jenkins parallel stages — build all 50 images simultaneously instead of sequentially.
- 3. Change detection:** Only rebuild images whose source code changed. Use `git diff --name-only` + image dependency map.
- 4. Optimized Dockerfiles:** Move `apt install` / `pip install` BEFORE `COPY . .` — ensures dependency layers are cached unless `requirements.txt` changes.
- 5. Multi-stage builds:** Remove dev tools from final image. Smaller image = faster push to ECR.
- 6. Self-hosted runners:** GitHub-hosted runners have 2-core, 7GB RAM. Self-hosted on c5.2xlarge (8 vCPU) = 3-4x faster builds.
- 7. Remote cache (BuildKit):** Use S3 or registry as shared build cache across runners:
`--cache-to=type=registry,ref=/cache`
- 8. Kaniko / BuildKit daemon:** Replace Docker-in-Docker with Kaniko in Kubernetes — better caching, no privileged container needed.

Expected gains:

Parallel builds alone: 20min -> ~5min. Add caching: ~5min -> ~2min. Change detection: ~2min -> <1min for unchanged services.

ApnaKart connection: Your Jenkins pipeline already uses ECR push. Mention adding `--cache-from=` in the docker build stage as a planned optimization.

Q7

How do you design a secure CI/CD pipeline with integrated DevSecOps practices?

Shift-left security layers:

- Pre-commit:** Secrets scanning (git-secrets, truffleHog, gitleaks). Prevents credentials from ever entering Git history.
- Code stage:** SAST with SonarQube — static analysis for code smells, SQL injection, XSS patterns. Quality gate blocks merge on critical issues.
- Build stage:** Dependency scanning (Trivy, Snyk) — scans `package.json`, `requirements.txt`, `pom.xml` for known CVEs.
- Image stage:** Container image scanning with Trivy — scans OS packages + app dependencies in final image. Block push if CRITICAL CVEs found.
- IaC stage:** Terraform security scanning with tfsec / Checkov — finds open security groups, unencrypted S3, missing MFA.
- Registry:** ECR Image Scanning on push — continuous scanning even after image is built.
- Deploy stage:** OPA/Gatekeeper admission controller — blocks deployment of images without valid scan digest or from unregistered registries.
- Runtime:** Falco runtime security — detects container escape attempts, exec into pods, privilege escalation at runtime.

Your ApnaKart pipeline has SonarQube + Trivy already — that's stages 2 and 4. Mention adding gitleaks at pre-commit and OPA at admission as next steps.

Q8

Pipeline works in staging but fails in production — how do you systematically debug?

Root cause categories (CEDAR framework):

Config differences: Env vars, feature flags, secrets — staging uses K8s Secrets while prod uses Vault? Different values for DB_HOST, REDIS_URL?

Environment scale: Resource limits too low in prod (prod has stricter limits). Readiness probe failing due to cold start under load.

Data differences: Prod DB has different schema version, missing migration. Data-driven bugs that don't surface on staging seed data.

Access/IAM differences: Service account has permission in staging but not in prod. ECR pull permission missing. S3 bucket policy.

Race conditions: Staging has 1 replica — no concurrency issues. Prod has 10 replicas — race condition surfaces.

Debug steps:

1. Compare ALL env vars: `kubectl get configmap/secret -n staging` vs `-n prod`
2. Compare resource limits and HPA config between environments
3. Check IAM role policies attached to the service account in prod
4. Run the exact failing step locally with prod-equivalent env vars
5. Add verbose logging temporarily to the failing step
6. Check if the issue is intermittent (retry logic, circuit breaker needed)

SECTION 3 — Cloud & Infrastructure

Q9

Design a multi-region, highly available architecture with failover and data consistency.

Architecture blueprint (Active-Active vs Active-Passive):

Active-Passive (simpler): Primary region serves traffic. Secondary region warm standby. Route 53 health check triggers failover. RTO: 2-5 min. RPO: seconds (with async replication).

Active-Active (complex, zero RPO): Both regions serve traffic. Route 53 latency-based or geolocation routing. Requires conflict-free data model.

Key components:

Route 53: Health checks + failover routing policy. TTL set to 60s for fast DNS propagation on failover.

CloudFront: Global CDN in front — continues serving cached content even if origin region is down.

RDS Aurora Global: Sub-second replication lag across regions. Promote secondary in <1 min. Handles read scaling automatically.

DynamoDB Global Tables: Multi-master, multi-region. Conflict resolution via last-write-wins. Ideal for session data, feature flags.

S3 Cross-Region Replication: Replicate assets, configs, and state backups. With S3 versioning enabled.

EKS in both regions: Separate clusters, same Helm chart + Kustomize overlay. ArgoCD syncs both from same Git repo.

Data consistency trade-off: CAP theorem: You choose CP or AP. For financial data: CP (Aurora Global). For user sessions: AP (DynamoDB Global Tables).

This maps directly to ApnaKart's architecture. Mention Route53 + CloudFront + Aurora as your foundation and 'multi-region as a planned next phase'.

Q10

Your AWS bill increased 3x overnight — how do you identify the root cause?

Immediate triage (first 30 minutes):

- AWS Cost Explorer -> Daily spend view -> Compare yesterday vs previous 7 days
- Service breakdown: Which service spiked? EC2? S3? Data Transfer? NAT Gateway?
- AWS Cost Anomaly Detection (if enabled) — sends SNS alert with root cause
- CloudTrail -> Filter last 24h -> Look for: large instance launches, S3 policy changes, new resources
- Trusted Advisor -> Cost optimization section

Common culprits and fixes:

NAT Gateway data processing: Every GB through NAT costs \$0.045. A logging misconfiguration pushing GB/s kills your bill. Fix: VPC endpoints for S3/DynamoDB.

EC2 forgotten instance: Someone launched a p3.8xlarge for ML test and forgot it. Enforce tag policies + billing alerts.

S3 egress from region: Data transfer out to internet is charged. Check S3 access logs for unexpected GetObject volume.

RDS storage autoscaling: DB storage hit limit, autoscaled 2TB -> 10TB. Set max storage limit.

EKS node autoscaling runaway: HPA + CA scaled to 100 nodes due to a metrics spike. Set CA max node count. Add PDB.

API Gateway / Lambda spike: Possible DDoS or bot traffic. Check CloudWatch request counts. Enable WAF.

Prevention: Set CloudWatch billing alarm at 150% of normal daily spend. Enable Cost Anomaly Detection. Tag all resources with team/project for attribution.

Q11

How do you handle infrastructure drift in Terraform across multiple teams?

Drift = real infra no longer matches Terraform state. Causes: manual console changes, API calls, partial applies.

Detection:

- terraform plan -detailed-exitcode in CI — exit code 2 means drift detected. Run as scheduled job (daily).
- driftctl (open source) — scans all AWS resources and reports unmanaged/drifted ones.
- AWS Config rules — detect out-of-compliance resource configs continuously.

Prevention:

- IAM policy: Deny all infrastructure changes except via Terraform service account (break-glass procedure for emergencies).
- Sentinel policies (Terraform Enterprise) or OPA (open source) — policy as code, block non-compliant plans.
- Remote state with locking (S3 + DynamoDB) — prevents concurrent applies causing divergence.
- Module versioning — pin module versions in all team configs. No floating refs.
- Atlantis or Terraform Cloud — GitOps for Terraform. All applies via PR. Full audit trail.

Your ApnaKart uses S3 + DynamoDB for remote state — that's the foundation. Mention adding scheduled 'terraform plan' in Jenkins to detect drift as a next step.

Q12

Terraform apply failed midway — how do you recover without corrupting state?

Understanding partial apply:

Terraform is not transactional. If apply fails at resource #7 of 20, resources 1-6 are created and in state, 7 may be partially created, 8-20 are not created.

Recovery steps:

1. **DO NOT re-run apply blindly:** First understand what succeeded and what failed. Check .tfstate file or terraform show.
2. **terraform refresh:** Sync state with actual cloud resources to get accurate picture of what exists.
3. **Investigate failed resource:** Check AWS console for the failed resource. Common: IAM permission denied, limit exceeded, dependency missing.
4. **Fix the root cause:** Update Terraform code or fix the permission/limit issue.
5. **Re-run terraform apply:** Terraform will skip already-created resources (they're in state) and only create/update remaining ones.
6. **If state is corrupted:** Use terraform state rm to remove corrupt entry, then terraform import to re-import the actual resource.
7. **State backup:** S3 versioning on state bucket means you can restore previous state: aws s3api list-object-versions --bucket

```
terraform state list # see what's in state
```

```
terraform state show aws_instance.web # inspect specific resource
```

```
terraform state rm aws_instance.bad # remove corrupt entry
```

```
terraform import aws_instance.web i-0abc123 # re-import from AWS
```

SECTION 4 — Security & Secrets

Q13

How do you design secure secrets management across 100+ microservices?

The hierarchy of secrets management maturity:

Level 0 (Anti-pattern): Hardcoded in code / env vars in Dockerfile. Never do this.

Level 1 (Basic): Kubernetes Secrets — base64 encoded, not encrypted at rest by default. OK for start, not scale.

Level 2 (Better): AWS Secrets Manager or Parameter Store — encrypted, versioned, rotation support, audit trail via CloudTrail.

Level 3 (Best for scale): HashiCorp Vault — dynamic secrets, fine-grained policies, PKI as a service, lease-based rotation.

For 100+ microservices — recommended architecture:

External Secrets Operator (ESO): Runs in Kubernetes. Syncs secrets from AWS Secrets Manager / Vault -> Kubernetes Secrets automatically. Services use K8s Secrets API but source of truth is external.

IRSA (IAM Roles for Service Accounts): Each microservice's pod gets an IAM role (not shared credentials). Least privilege per service.

Secret rotation: AWS Secrets Manager auto-rotates DB passwords. ESO re-syncs to K8s within minutes. Zero manual intervention.

Audit trail: CloudTrail logs every GetSecretValue call — who/what accessed which secret and when.

Vault dynamic secrets: Vault generates short-lived DB credentials on demand — each pod gets unique credentials that expire. Massive blast radius reduction.

Identified gap in ApnaKart: Kubernetes Secrets stored as base64 (Level 1). Mention plan: migrate to External Secrets Operator + AWS Secrets Manager as your current learning focus.

Q14

Your system passed all scans but got compromised — what security layers were missing?

The 'all green but still breached' scenario — common missing layers:

Runtime security (zero-day): Scans catch known CVEs. A zero-day exploit in a library won't appear in Trivy/Snyk. Need Falco for runtime anomaly detection.

Supply chain attack: Your dependencies were clean — but a dependency's dependency was compromised (SolarWinds-style). Need SBOM + sigstore/cosign image signing.

Lateral movement controls: Once inside one pod, attacker moved to others. Missing: NetworkPolicy deny-all + allow-list. Missing: mTLS between services.

RBAC over-permission: Service account had cluster-admin. Attacker used it to exfiltrate secrets from all namespaces. Principle of least privilege.

Secrets in environment variables: Scans don't find secrets injected at runtime as env vars. Attacker read /proc/environ or app debug endpoint.

Ingress without WAF: OWASP Top 10 attacks (SQL injection, SSRF) bypassed application-level security. Need AWS WAF in front of ALB.

Missing egress controls: Attacker exfiltrated data via outbound HTTP to external C2 server. NetworkPolicy should whitelist-only required egress IPs.

Unpatched node OS: Container escape via kernel vulnerability (runc CVE). Node OS patching schedule missing.

This is a 'defense in depth' question. The answer shows your understanding that security is layered — scanning is just one layer. Runtime, network, RBAC, and supply chain are equally critical.

SECTION 5 — Observability & Reliability

Q15

How do you design observability (logs, metrics, traces) for a distributed system?

The Three Pillars + the fourth signal:

Metrics (RED + USE): Rate, Errors, Duration per service (RED). Resource Utilization, Saturation, Errors per node (USE). Prometheus + Grafana. Pre-built dashboards per service.

Logs (structured): JSON-formatted logs with correlation ID, service name, trace ID, environment. Centralized in ELK or CloudWatch Logs Insights. NEVER use `log.info('user: ' + name)` — use structured fields.

Traces (distributed): OpenTelemetry auto-instrumentation -> Jaeger or Datadog APM. Trace spans across microservices to find bottleneck. Sampling: 100% for errors, 1% for normal traffic.

Events (fourth signal): Deployment events, scaling events, config changes overlaid on dashboards. When latency spiked at 2pm — was there a deploy at 1:58pm?

Correlation ID pattern (critical):

Generate UUID at API Gateway entry. Inject as header X-Correlation-ID. Each service passes it downstream and includes in all log entries. This lets you trace a single user request across 10 microservices in one log query.

Log query example:

```
fields @timestamp, service, message | filter correlationId='abc-123' | sort @timestamp asc
```

ApnaKart uses Datadog APM + Prometheus + Grafana. Mention the correlation ID strategy as something you implemented or plan to implement for distributed tracing.

Q16

How do you implement SLO-based alerting without alert fatigue?

SLI -> SLO -> Error Budget -> Alert on burn rate:

SLI (Service Level Indicator): Measured metric. e.g., 'fraction of HTTP requests returning 2xx in under 200ms'

SLO (Service Level Objective): Target. e.g., '99.9% of requests successful over 30 days'

Error Budget: 0.1% of 30 days = 43.2 minutes of allowed downtime/month.

Burn rate alerting (Google SRE model):

Fast burn (page now): Burn rate > 14x in 1h window. You'll exhaust error budget in 2h. Wake up on-call immediately. Multi-window: 1h + 5min.

Slow burn (ticket): Burn rate > 3x in 6h window. You'll exhaust budget in ~5 days. Create incident ticket, don't page.

No alert zone: Burn rate < 1x. You're well within budget. No alert needed — this eliminates noise.

Anti-patterns causing alert fatigue:

- Alerting on symptoms not causes (CPU > 80% instead of error rate increasing)
- Too-tight thresholds with no hysteresis (alert fires, recovers, fires again every minute)
- Alerting at static thresholds — no time-of-day awareness
- Every alert goes to same channel — no severity-based routing
- No runbook linked to alert — on-call doesn't know what to do

Prometheus recording rules make burn rate calculation efficient — pre-compute error ratios instead of querying raw metrics in alert rules.

Q17

Service latency spikes every 60 seconds — how do you debug root cause?

60-second periodicity is a huge diagnostic clue. Think: what runs every 60 seconds?

Kubernetes liveness probe: Default period is 10s but often set to 60s. If probe temporarily fails, pod gets killed/restarted causing latency.

Prometheus scrape interval: Default 60s scrape. If app generates metrics under heavy load, scrape can add latency to the instrumented handler.

Cron job / scheduled task: A database backup, log rotation, or aggregation job running every minute consuming I/O.

Connection pool refresh: Some drivers refresh connections every 60s — during refresh, requests queue up.

GC (Garbage Collection): JVM G1GC full GC triggered periodically. Check: -XX:+PrintGCDetails logs, jstat -gcutil.

HPA evaluation cycle: HPA evaluates every 15s by default but scaling events may cluster at 60s due to cooldown periods.

Kubernetes token refresh: Service account token TTL refresh causing brief auth delays.

External dependency polling: A dependency being polled every 60s with high latency response.

Debug method:

1. Graph latency P99 in Grafana with 1s resolution — confirm the 60s periodicity precisely
2. Overlay: probe_duration_seconds, GC pause metrics, cron job completion time, HPA events
3. Use Datadog APM waterfall to see which span in the trace is slow during the spike
4. strace or perf on the container process during a spike window

SECTION 6 — Production & Incident Handling

Q18

Production is down — what are your first 5 steps under pressure?

The STAR incident response framework:

- 1. Acknowledge + Communicate (0-2 min):** Post in #incidents Slack: 'Production issue detected, investigating. Updates every 10 min.' Stakeholders need to know someone is on it — silence is worse than 'investigating'.
- 2. Establish blast radius (2-5 min):** Is it 100% down or partial? Which services/regions/customers affected? `kubectl get pods -A | grep -v Running`. Check dashboards.
- 3. Check recent changes (5-8 min):** `git log --oneline -10`. ArgoCD deployment history. AWS CloudTrail. 80% of outages are caused by recent changes.
- 4. Rollback if possible (8-15 min):** If a recent deploy caused it: `kubectl rollout undo deployment/`. Or redeploy previous Helm release: `helm rollback`. Don't investigate while users are down — rollback first.
- 5. Isolate and investigate (15+ min):** If rollback doesn't fix: collect logs, traces, metrics around the start time. Work hypothesis tree: infra layer -> service layer -> data layer.

Critical: Separate the 'restore service' track from the 'find root cause' track. Don't let root cause analysis block service restoration.

Q19

How do you design systems for graceful degradation during failures?

Graceful degradation = system stays partially functional instead of fully failing:

- Circuit Breaker (Resilience4j / Hystrix):** After N consecutive failures to a downstream service, open the circuit — return cached/default response instead of waiting for timeout. Auto-retry after cooldown period.
- Fallback responses:** If recommendation service is down, return empty list rather than 500 error. If payment gateway timeout, show 'try again in 5 min' instead of error page.
- Timeout hierarchy:** Set aggressive timeouts at each service boundary. No timeout = one slow service takes down the whole call chain. Rule: outermost timeout > sum of inner timeouts.
- Bulkhead pattern:** Separate thread pools for different downstream services. If DB thread pool is exhausted, it doesn't block API thread pool. Isolate failure domains.
- Rate limiting + load shedding:** Under extreme load, return 429 to lowest-priority users rather than degrading service for all. Preserve SLO for premium users.
- Read replicas for read traffic:** During primary DB degradation, route reads to replica. Accept stale reads rather than serving errors.
- Feature flags (kill switches):** LaunchDarkly / AWS AppConfig. Disable non-critical features (recommendations, social feed) under load to protect core checkout flow.
- Async over sync:** Move non-critical operations to queue (SQS). Email confirmation doesn't need to block checkout response.

Mention ApnaKart: Redis caching is a form of graceful degradation — if DB is slow, cached responses serve users. Mention SQS for order confirmations as async pattern.

Q20

How do you ensure zero-downtime cluster upgrades in Kubernetes?

EKS control plane upgrade is AWS-managed and non-disruptive. Worker node upgrade is YOUR responsibility:

- 1. Upgrade control plane first:** AWS EKS: `eksctl upgrade cluster --name --version 1.29 --` Control plane upgrades are in-place, no downtime.

2. Update add-ons: Upgrade CoreDNS, kube-proxy, VPC CNI to versions compatible with new k8s version before upgrading nodes.

3. PodDisruptionBudgets: Ensure all critical workloads have PDB (minAvailable >= 1). Without PDB, node drain can kill all replicas simultaneously.

4. Node group rolling upgrade: Create new node group with new AMI/k8s version. Taint old nodes. Let new nodes register. Drain old nodes one by one.

5. kubectl drain --ignore-daemonsets: Gracefully evicts pods from node. Respects PDB. Sets node as unschedulable. Pods reschedule on new nodes.

6. Managed node groups (EKS): eksctl upgrade nodegroup -- AWS handles drain, terminate, replace automatically. Zero-downtime if PDB is set.

7. Verify and clean up: kubectl get nodes — verify all new nodes are Ready. Delete old node group. Validate all pods running.

Common mistakes that cause downtime:

- No PDB set — drain kills all pods of a service simultaneously
- Resource requests not set — pods can't reschedule (pending) on new nodes without requests
- Node affinity hardcoded to specific node name — pods stuck
- Upgrading more than 1 minor version at a time (e.g., 1.26 -> 1.29 direct) — unsupported

Your ApnaKart runs on EKS — you can walk through this upgrade process with real context. Mention managed node groups and PDB configuration as what you'd do first.